



Khulna University of Engineering and Technology

Department of Computer Science and Engineering

Course Title: Software Engineering & Project Management

Course Code: CSE 3220

Project Title: Rento

Submission Date: 5 April, 2026

Live URL: <https://rento-e8w1.onrender.com/>

Repository: <https://github.com/commoner02/Rento>

Submitted to:

Safin Ahmmed

Lecturer, Department of Computer Science and Engineering

Nabil Faiyaz Sadi

Lecturer, Department of Computer Science and Engineering

Submitted by:

Name	Roll
Sree Shuvo Kumar Joy	2107116
Saleheen Uddin Sakin	2107103

Group: B2 | Year: 3rd | Semester: 2nd

1. Project Overview

Rento is a full-stack rental marketplace for party and event equipment. Users can browse products, create rental orders, and manage the complete rental lifecycle. The project demonstrates professional software engineering practices — clean layered architecture, role-based security, automated testing, containerisation, and continuous deployment.

2. Technology Stack

Category	Technology
Backend	Java 17 · Spring Boot 3.2 · Spring MVC · Spring Security
View Layer	Thymeleaf (server-side rendering)
Database	PostgreSQL 15 · Spring Data JPA / Hibernate
Testing	JUnit 5 · Mockito · MockMvc · H2 (in-memory, test)
Build	Maven 3.8
DevOps	Docker · Docker Compose · GitHub Actions · Render

3. Roles

Role	Permissions
ADMIN	Manage products, manage users & roles, view all rentals and payments, process refunds
SELLER	Confirm / activate rentals, process returns, log damage fees
BUYER	Browse products, create rentals, view own rentals, cancel own pending rentals

4. System Architecture

Rento follows a standard Spring MVC layered architecture. Each layer communicates only with its immediate neighbour, ensuring low coupling and high cohesion.

Layers (top to bottom):

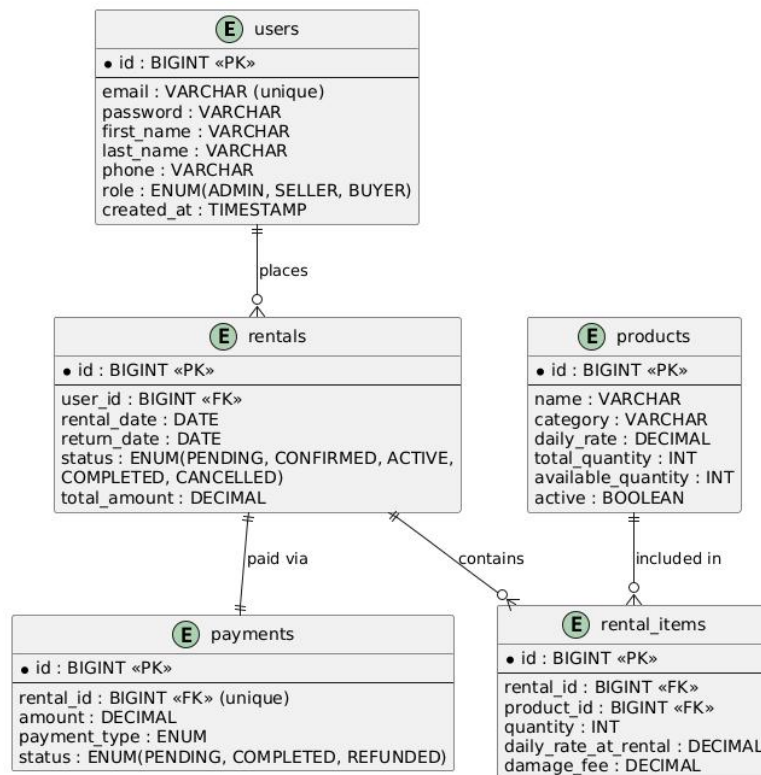
- Presentation Layer — Thymeleaf templates
- Controller Layer — AuthController • ProductController • RentalController • AdminController
- Service Layer — UserService • ProductService • RentalService (business logic)
- Repository Layer — Spring Data JPA interfaces (no hand-written SQL)
- PostgreSQL 15 — 5 tables

Controllers accept HTTP requests and delegate to services. Services hold all business rules.

Repositories provide JPA interfaces. DTOs decouple the HTTP boundary from domain entities.

5. Database Design

5.1 Entity-Relationship Diagram

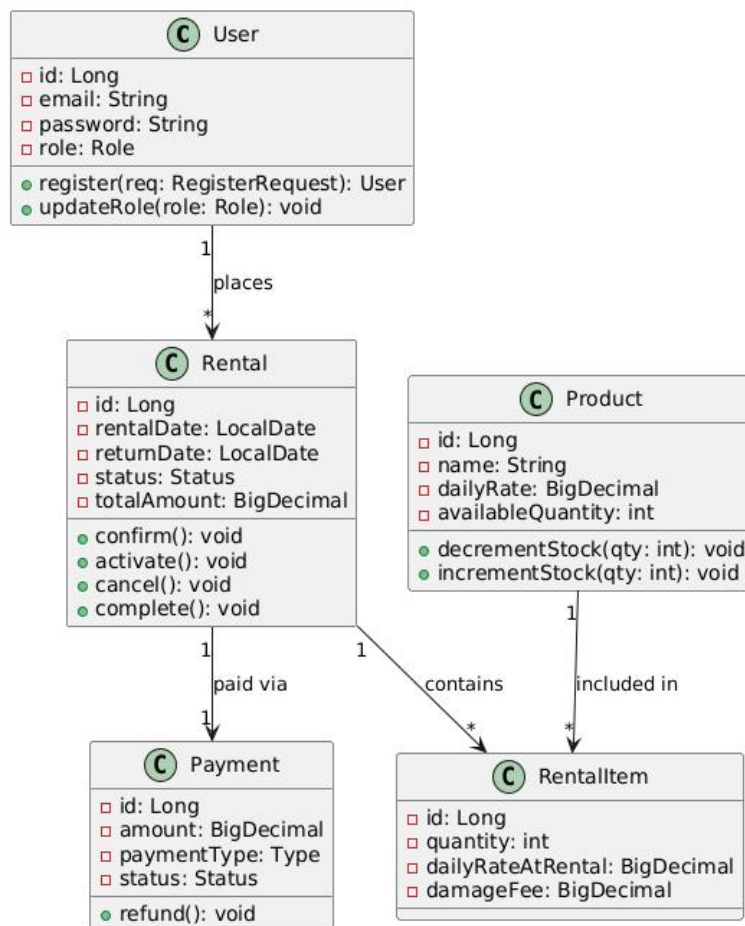


5.2 Relationships

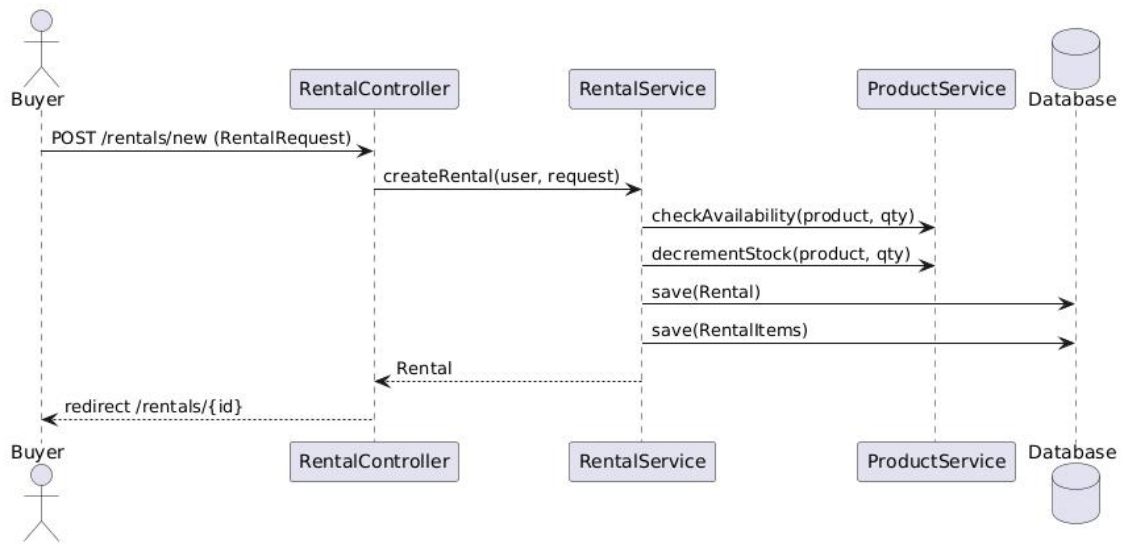
From	Cardinality	To	FK Column
users	1 : M	rentals	rentals.user_id
rentals	1 : M	rentalItems	rentalItems.rental_id
products	1 : M	rentalItems	rentalItems.product_id
rentals	1 : 1	payments	payments.rental_id
users	M : M	products	via rentals

6. UML Diagrams

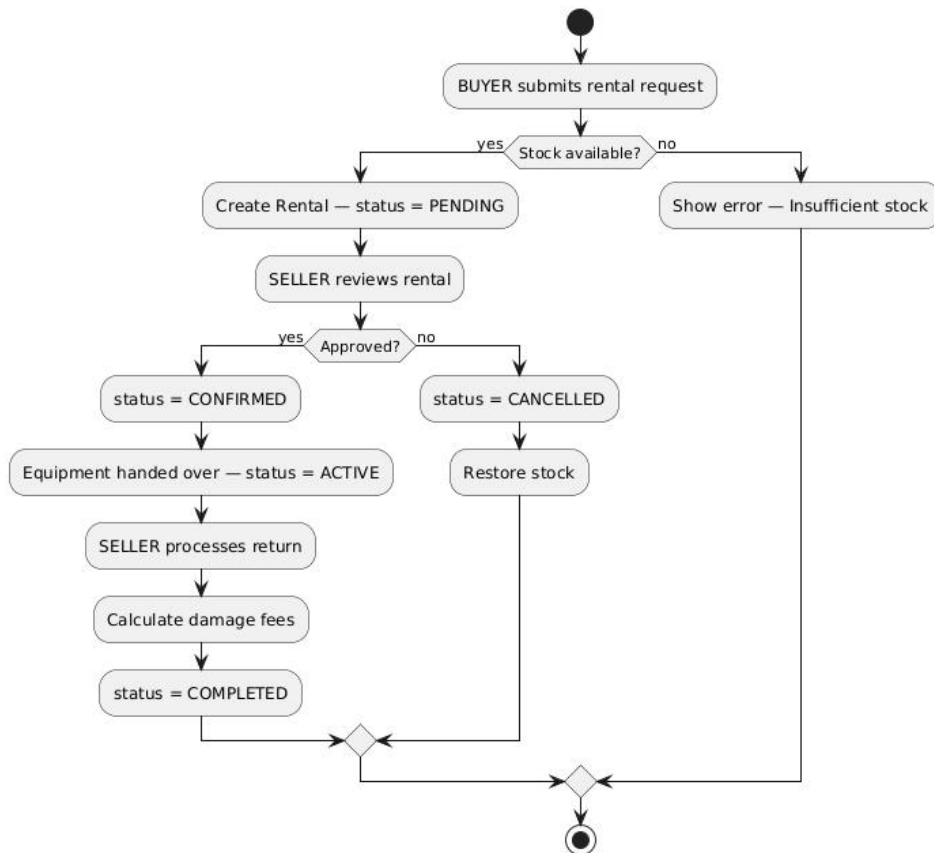
6.1 Class Diagram



6.2 Sequence Diagram — Create Rental



6.3 Activity Diagram — Rental Lifecycle



7. Security

Spring Security is configured in SecurityConfig.java with URL-based rules as the first defence layer and method-level annotations as a second layer. Passwords are encoded with BCryptPasswordEncoder (strength 10).

Endpoint Pattern	ADMIN	SELLER	BUYER	Public
/auth/**	✓	✓	✓	✓
/products (GET)	✓	✓	✓	–
/products (POST/DELETE)	✓	–	–	–
/rentals/new (GET/POST)	–	–	✓	–
/rentals/*/confirm activate	✓	✓	–	–
/rentals/*/return	✓	✓	–	–
/admin/**	✓	–	–	–

8. REST API Endpoints

Authentication

Method	URL	Access	Description
GET	/auth/login	Public	Login page
POST	/auth/login	Public	Process login
GET	/auth/register	Public	Register page
POST	/auth/register	Public	Create BUYER account

Products

Method	URL	Access	Description
GET	/products	Auth	List all products
GET	/products/{id}	Auth	Product detail
POST	/products/new	ADMIN	Create product
POST	/products/edit/{id}	ADMIN	Update product
POST	/products/delete/{id}	ADMIN	Delete product

Rentals

Method	URL	Access	Description
GET	/rentals	Auth	List rentals (role-filtered)
POST	/rentals/new	BUYER	Create rental
POST	/rentals/{id}/confirm	SELLER/ADMIN	Confirm rental
POST	/rentals/{id}/activate	SELLER/ADMIN	Mark active
POST	/rentals/{id}/cancel	Owner/SELLER/ ADMIN	Cancel rental
POST	/rentals/{id}/return	SELLER/ADMIN	Process return

Admin

Method	URL	Access	Description
GET	/admin/users	ADMIN	List all users
POST	/admin/users/{id}/role	ADMIN	Change user role
GET	/admin/payments	ADMIN	View all payments
POST	/admin/payments/{id}/refund	ADMIN	Refund payment

9. Testing

35+ tests across three layers, all running against an H2 in-memory database in CI — no external dependencies required.

Test Class	Type / Count	Coverage
UserServiceTest	Unit • 4	register, duplicate email, findByEmail, updateRole
ProductServiceTest	Unit • 8	CRUD operations, stock management, unavailable product
RentalServiceTest	Unit • 7	create, confirm, activate, cancel, processReturn
ProductControllerTest	Integration • 5	HTTP status codes, role-gating, form submission
RentalControllerTest	Integration • 6	Full request lifecycle, auth checks, redirects
RentalControllerSecurityTest	Security • 5	Unauthenticated access, role enforcement

10. Docker & CI/CD

10.1 Docker

A two-stage Dockerfile keeps the production image lean: the Maven build stage compiles the JAR, and only the JRE-Alpine runtime image (with the JAR) is shipped. docker-compose.yml orchestrates two services: postgres:15-alpine and the application, with a healthcheck ensuring the app only starts after the database is ready.

```
# Stage 1 — Build
FROM maven:3.9-eclipse-temurin-17 AS builder
RUN mvn clean package -DskipTests
# Stage 2 — Runtime
FROM eclipse-temurin:17-jre-alpine
COPY --from=builder /app/target/rento-*.jar app.jar
ENTRYPOINT ["java", "-jar", "app.jar"]
```

10.2 CI/CD Pipeline (GitHub Actions)

Triggers on every push to develop/main and on pull requests to main. Deployment to Render only occurs on main branch pushes.

#	Step	Detail
1	Checkout	actions/checkout@v4
2	Setup JDK	Java 17 Temurin with Maven cache
3	Run Tests	mvn clean test (H2, SPRING_PROFILES_ACTIVE=test)
4	Build JAR	mvn package -DskipTests
5	Deploy	curl to Render API (main branch only, uses secrets)

10.3 Git Workflow

- main — protected, no direct push; PR + 1 review approval required
- develop — integration branch; feature branches merge here first
- feature/* — individual feature branches (user-auth, product-crud, rental-flow, admin-panel)

10.4 Deployment

Deployed on Render. Build command: mvn clean package -DskipTests. Start command: java -jar target/rento-*.jar. Environment variables (DATABASE_URL, DB_USERNAME, DB_PASSWORD) injected via Render dashboard — never hardcoded.